

Homework 5

CS/ECE 374-B Intro to Algorithms & Models of Computation

University of Illinois Urbana-Champaign

Spring 2025

Instructors: Abhishek Umrawal & Ivan Abraham

Maximum Points: 100

Due Date: Friday, April 4, 2025, 11:59 PM

Instructions

1. Answer **ALL** questions.
2. Show all the work.
3. Legibly scribe your solutions and upload them as individual problems on Gradescope.

Suggestions

While writing your answers and/or proofs, think about the following.

1. What is it that you are given? What is it that you are assuming? What is it that you are trying to prove?
2. How exactly does a step follow from the previous step? Assume nothing is apparent, even algebraic simplifications. Your reasoning should be clear and convincing.
3. If you use a result from the class, state it clearly before using it. You should identify where exactly you need that result, cast your problem into the setup of the result, and then use it.
4. If you use a result from outside the class, you should prove it before using it.

While writing and analyzing an algorithm, note the following.

1. An algorithm is a finite set of primitive and unambiguous instructions to solve a computing problem.
2. Unless otherwise stated, you should give the fastest possible algorithm, i.e., an algorithm with the tightest possible asymptotic time complexity bound.

Instructions while using Dynamic Programming

While using dynamic programming to solve a problem, note the following.

1. Your solution must include a *recurrence* relation.
2. Your solution must include a brief description of what your recurrence relation and each of its cases represent.
3. Your solution must include a brief description of the *memory element/storage* and how it is filled in.
4. Your solution must include a *runtime* analysis.
5. Any recursive or dynamic programming algorithm without an English description of the recursive function, i.e., a description of the output of the function in terms of its inputs, will receive **no** credit.
6. **Greedy algorithms require proofs of correctness.** If you choose to design a greedy algorithm, you must prove that your algorithm is correct, or you will receive no credit, despite your algorithm being correct. This means you should check: ‘Are you absolutely sure you do not want to use dynamic programming instead?’

Problems

1. A diagonal of a polygon is defined to be a line segment between two vertices that do not form an edge of the polygon. A polygon is *convex* if every diagonal lies entirely inside the polygon. If we cut a convex polygon along one of its diagonals, it will be partitioned into two smaller convex polygons.

Suppose we are given a convex polygon P with an even number of vertices labeled as $p_1, \dots, p_{2n}$, where $n \geq 2$. Describe and analyze an algorithm to compute the diagonals of a polygon P that partition it into convex quadrilaterals, where the area of the smallest quadrilateral is as large as possible. To make the question easier, you may choose either to let your function return the actual diagonals partition **or** just the area of the smallest quadrilateral under that partition.

Hint: Why do we assume the input has an even number of vertices? Is a partition into quadrilaterals always possible?

You may assume having access to an oracle $\text{Area-of-Quad}(i, j, k, l)$ that returns the area of the quadrilateral with vertices p_i, p_j, p_k, p_l in $O(1)$ time. One can trivially implement this function with the right math¹, but you do not need to know anything about that implementation to design your algorithm.

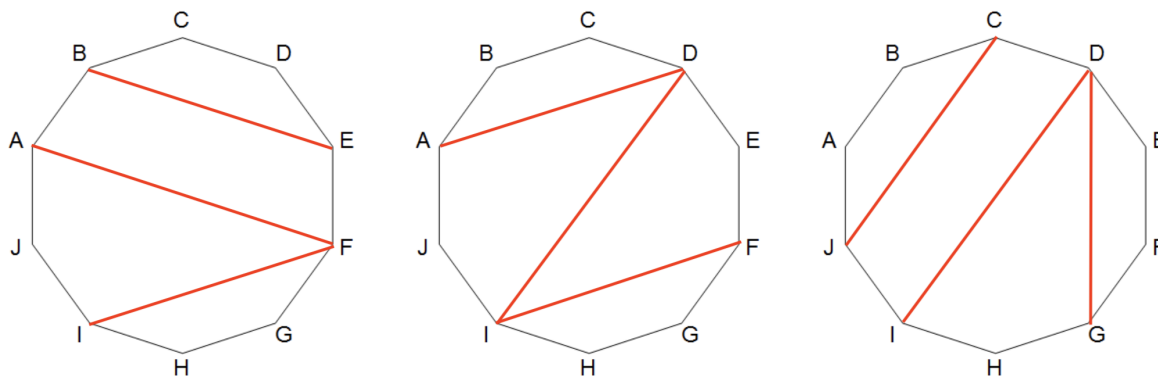


Figure 1: Different ways to partition a convex decagon, $P = \{A, B, \dots, J\}$, into four convex quadrilaterals.

¹ $\text{Area-of-Quad}(i, j, k, l) = x_i y_j - x_j y_i + x_j y_k - x_k y_j + x_k y_l - x_l y_k + x_l y_i - x_i y_l$

Solution: We want to compute the optimal quadrilateral partition using divide and conquer. It could be easily proved using induction that there is at least one valid way of doing such a quadrilateral partition for any given convex polygon P with an even number of vertices. Similarly, there will be no valid quadrilateral partition for a convex polygon P with an odd number of vertices. Thus, we can try and guess a possible quadrilateral in the quadrilateral-partition, and then recurse on the rest of the polygons it creates. The problem we need to notice is to cut such a quadrilateral in a way that we can represent the recursive sub-problems in an efficient/ succinct way.

Assume that the polygon P' , with an even number of vertices, is specified as a list of vertices $i \dots j$ in a clockwise direction. With some attention, we notice that in any quadrilateral partition of P , there exists one quadrilateral that uses the edge between vertex i and j . To be more detailed, removing any quadrilateral using the edge between i and j leaves us with up to three polygons whose vertices are consecutive along the original polygon. Let $\text{MaxSmall}[i, j]$ denote the maximized area for the smallest quadrilateral among all valid partitions for a polygon starting at vertex i and ending at vertex j , inclusively. We have the following recurrence:

$$\text{MaxSmall}[i, j] = \begin{cases} \infty, & \text{if } j - i = 1, \\ 0, & \text{else if } j - i + 1 \text{ is odd,} \\ \max_{i < k < t < j} (\text{MinArea}), & \text{otherwise.} \end{cases}$$

where,

$$\text{MinArea} = (\min\{\text{Area-of-Quad}(i, k, t, j), \text{MaxSmall}[i, k], \text{MaxSmall}[k, t], \text{MaxSmall}[t, j]\}).$$

In the recursion, if $\text{MaxSmall}[i, j]$ is an invalid representation or a representation of a convex containing an odd number of vertices, we will return 0 as there won't be any valid partition. If the convex only has 4 vertices, we will just return the area of this quadrilateral, as there is no further partition. If there are more than 4 vertices, we first pick vertex i and j , as well as any other two distinct vertices k and t where $i < k < t < j$. We will cut the quadrilateral formed by (i, k, t, j) , and then we are left with three new sub-problems, and we will keep recursing. We will return the maximized smallest area among all valid choices of k and t .

Notice that edges in the original polygon are excluded in the return value, as all edges will appear in any valid quadrilateral partition. To get this, the actual way of doing the partition that maximizes the smallest quadrilateral, we can have another recursion:

$$\text{MaxSmallArg}[i, j] = \begin{cases} \emptyset, & \text{if } j - i \leq 2, \\ \emptyset, & \text{if } j - i + 1 \text{ is odd,} \\ \emptyset, & \text{if } j - i = 3, \\ \text{every edge in the quadrilateral } (i, k', t', j) & \\ \text{that is a diagonal in the original } P, & \text{otherwise.} \\ \text{where } (t', k') = \arg \max_{[i, j]} \text{MaxSmall}[i, j] & \end{cases}$$

In either recursion, there is a total of $O(n^2)$ different cases, we can have 1/2 2-D array tables: MaxSmallValue and MaxSmallArg for memoization, where we will be filling the tables in decreasing row order and increasing column order.

The time it takes to fill one case is $O(n^2)$, and we have a total of $O(n^2)$ cases. So the cost for the algorithm is $O(n^4)$ in runtime and $O(n^2)$ in space. The value we are interested will be $\text{MaxSmall}(1, 2n)$ or $\text{MaxSmallArg}(1, 2n)$.

2. Given an integer array $\mathbf{x}[1, \dots, n]$, design and analyze an algorithm that finds a *contiguous* sub-array of $\mathbf{x}[1, \dots, n]$ such that the sum of its elements is the largest among all such sub-arrays. To receive full credit, your algorithm must run faster than $O(n^2)$.

Solution: Refer to Algorithm 1. It is a divide-and-conquer algorithm that keeps dividing the problem into two halves and processes each subproblem recursively. The main function is $\text{MaxSubarraySum}(\text{arr}, \text{left}, \text{right})$ that calls the subroutine $\text{MaxCrossingSum}(\text{arr}, \text{left}, \text{mid}, \text{right})$. Calling $\text{MaxSubarraySum}(\mathbf{x}, 1, n)$ will give the solution to the original problem.

The recurrence relation for the runtime of the above algorithm for calling $\text{MaxSubarraySum}(\mathbf{x}, 1, n)$ is $T(n) = 2T(n/2) + O(n)$. By solving the recurrence, the asymptotic runtime bound is $O(n \log n)$.

Algorithm 1 Find Maximum Subarray Sum using Divide and Conquer

3. Consider the following scenario. Your friendly neighborhood TA Tue is planning a party for his arbitrarily large company of size n . The company is steeped in tradition, and has a hierarchy structure that resembles a tree, where every employee (besides the big boss) has one superior and some number of underlings with no cycles in the chain of command. To avoid potential awkwardness, Tue decides that direct superiors/underlings are not to be invited to the party together; that is, no neighbors in the hierarchy are invited together. With prior knowledge and observation skills, Tue has a preordained numerical score of how fun each member of the company would be at a party. Help Tue compute the score of the set of invitations that maximizes the fun score of the party, by describing and analyzing an algorithm to do so.

Solution:

$$\begin{aligned} \text{MaxFun}(\emptyset) &= 0 \\ \text{MaxFun}(v) &= \max \left\{ \begin{array}{l} \text{fun}(v) + \sum_{\text{grand-underling } x \text{ of } v} \text{MaxFun}(x) \\ \sum_{\text{underling } w \text{ of } v} \text{MaxFun}(w) \end{array} \right. \end{aligned}$$

Since our hierarchy is in the form of a tree, we have a natural order for our dynamic programming structure. We only need to make sure we have solved all the underlying (underling) problems before we combine the results to evaluate the maximum fun at any node. Then taking a postorder traversal/topological sort in reverse order/etc. enumeration of the nodes, we calculate the maximum order for each node from the leaves to the roots of the tree. **Any solution that does not explicitly mention traversal order results in a significant deduction!** We visit each node once, and the recurrence only consists of a comparison that takes $O(1)$ time with our memoization, so we have that the algorithm runs in $O(n)$ time.

4. Suppose you are taking an exam that does not give you enough time to solve all the questions. Let $\mathbf{t}[i]$ be the time taken in solving the question $\mathbf{a}[i]$ that is worth $\mathbf{p}[i]$ points, $i = 1, \dots, n$. Given three arrays, questions, $\mathbf{q}[1, \dots, n]$, times, $\mathbf{t}[1, \dots, n]$, and points, $\mathbf{p}[1, \dots, n]$, design and analyze an algorithm to decide what questions to solve on an exam with total time $\tilde{t}$ such that the total points received on the exam is maximized. You can assume that attempting a question means you will be able to solve it, and there is no partial credit.

Solution:

$$\text{MaxPoint}(i, \tilde{t}) = \begin{cases} 0, & \text{if } i > n \text{ or } \tilde{t} \leq 0, \\ \text{MaxPoint}(i-1, \tilde{t}), & \text{if } i \leq n \text{ and } \mathbf{t}[i] \geq \tilde{t}, \\ \max(\text{MaxPoint}(i-1, \tilde{t}), \mathbf{p}[i] + \text{MaxPoint}(i-1, \tilde{t} - \mathbf{t}[i])) & \text{otherwise.} \end{cases}$$

$\text{MaxPoint}(i, \tilde{t})$ returns the maximal points possible with only questions $\mathbf{q}[1, \dots, i]$ if given $\tilde{t}$ minutes.

We use a $n \times \tilde{t}$ matrix as the memorization structure, where n is the number of questions and $\tilde{t}$ is the total time of the exam (in minutes).

Observing the dependency of parameters, we know we need to fill the memorization structure from left to right, top to bottom (assuming top left is $(0, 0)$). Refer to Algorithm 2.

Algorithm 2 $\text{MaxPoint}(\mathbf{t}[1..n], \mathbf{p}[1..n], \tilde{t})$

```

1: Define  $\mathbf{M} = \text{Matrix}[0, \dots, n; 0, \dots, \tilde{t}]$ 
2: for  $j$  from 0 to  $\tilde{t}$  do
3:    $M[0, j] := 0$ 
4: for  $i$  from 1 to  $n$  do
5:    $M[i, 0] := 0$ 
6: for  $i$  from 1 to  $n$  do
7:   for  $j$  from 1 to  $\tilde{t}$  do
8:     if  $\mathbf{t}[i] > j$  then
9:        $M[i, j] := M[i-1, j]$ 
10:    else
11:       $M[i, j] := \max(M[i-1, j], M[i-1, j - \mathbf{t}[i]] + \mathbf{p}[i])$ 
12: return  $M[n, \tilde{t}]$ 

```

To find the subset of problems that gives the maximal points, we do a second iteration over the memorization matrix:

- Start with the bottom right cell (the cell with the highest value).
- If the cell above has the same value, it means that this row's item is not included, and we go to the cell above.
- If the cell above has a different value, it means that the current row's item is included, and we move to the row above, and we move to the left as many times as the weight of the included item.
- Continue to do steps b and c until a cell with value 0 is found.

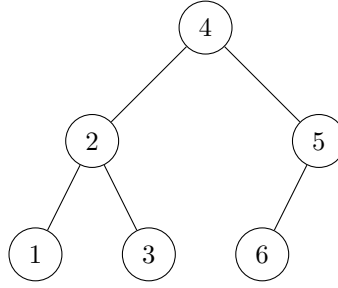
For a graphic explanation, please see: https://www.w3schools.com/dsa/dsa_ref_knapsack.php

The memorization structure takes $O(nt)$ space and each space takes $O(1)$ time to fill, so the total time complexity is $O(nt)$.

5. Binary trees can be traversed in multiple ways, with *inorder* and *postorder* traversals being two common methods explained as follows.

- Inorder Traversal: Nodes are visited in the order: Left child $\rightarrow$ Node $\rightarrow$ Right child.
- Postorder Traversal: Nodes are visited in the order: Left child $\rightarrow$ Right child $\rightarrow$ Node.

For example, consider the following binary tree.



The inorder and postorder traversals of the above tree are as follows.

- Inorder Traversal: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6 \rightarrow 5$.
- Postorder Traversal: $1 \rightarrow 3 \rightarrow 2 \rightarrow 6 \rightarrow 5 \rightarrow 4$.

Given two integer arrays, `inorder[1, ..., n]` and `postorder[1, ..., n]`, representing the inorder and postorder traversals of a binary tree with n nodes, design and analyze an algorithm to reconstruct the original binary tree. You may assume that all nodes are *distinct*, i.e., each node is represented by different symbols.

Hint: This can be efficiently achieved using a *divide-and-conquer* approach. Think about how to locate the root.

Solution: To reconstruct the binary tree, we use a recursive divide-and-conquer approach:

- First, we find the root node by taking the last node from `postorder`, `postorder[n]`. Find its location m in `inorder`, where `inorder[m] = postorder[n]`.
- We utilize m to find the left and right subtrees
 - Left subtree: `inorder[1, ..., m - 1]` and `postorder[1, ..., m - 1]`
 - Right subtree: `inorder[m + 1, ..., n]` and `postorder[m, ..., n - 1]`
- Recursively reconstruct the left and right subtrees using the corresponding segments of the inorder and postorder arrays.

Algorithm 3 Reconstruct Binary Tree from Inorder and Postorder Traversals

```

1: procedure RECONSTRUCTTREE(inorder, postorder)
2:   if postorder is empty then
3:     return NULL
4:    $root \leftarrow postorder[n]$ 
5:   Create a new node with value  $root$ 
6:   Find index  $m$  such that inorder[m] = root
7:    $left\_inorder \leftarrow inorder[1, \dots, m - 1]$ 
8:    $right\_inorder \leftarrow inorder[m + 1, \dots, n]$ 
9:    $left\_postorder \leftarrow postorder[1, \dots, m - 1]$ 
10:   $right\_postorder \leftarrow postorder[m, \dots, n - 1]$ 
11:   $node.left \leftarrow RECONSTRUCTTREE(left\_inorder, left\_postorder)$ 
12:   $node.right \leftarrow RECONSTRUCTTREE(right\_inorder, right\_postorder)$ 
13:  return  $node$ 

```

For the complexity of this algorithm, let n be the number of nodes in the tree. Since we need to find the location of the root node for each call of the function, we may choose a data structure efficient in searching (e.g. hashmap). Constructing the hashmap takes $O(n)$ time, while each lookup takes an amortized $O(1)$ time. The recursive function processes each node once, resulting in $O(n)$ time complexity. Thus, the overall time complexity is amortized $O(n)$, and the space complexity is $O(n)$ due to recursion stack and hashmap storage.